

Codex CLI 첫 실습 인쇄 매뉴얼

준비물 없이 시작하는 2시간 따라하기 수업 | Node.js, Codex CLI, Plan Mode, 미니 프로젝트, 테스트, GitHub 업로드

오늘의 핵심

사람은 주제와 기준을 정하고, Codex는 계획, 구현, 수정, 검증, 업로드를 돕는다.

코드를 직접 편집하지 않는다. 대신 Codex에게 정확히 요청하고 결과를 확인한다.

최종 목표는 작은 정적 웹 프로젝트를 브라우저에서 확인하고 테스트까지 통과시키는 것이다.

1. 오늘 만들 결과물

각 참가자는 자기 주제로 작은 정적 웹 미니 프로젝트를 만든다. 공통 파일 구조는 다음과 같다.

README.md	프로젝트 목적, 실행 방법, 확인 방법을 설명한다.
index.html	브라우저에 보이는 화면 구조를 담는다.
style.css	색상, 간격, 글자 크기 같은 디자인을 담당한다.
script.js	버튼 또는 입력창 동작을 담당한다.
test.js	파일 존재, 연결 상태, 기본 구성 여부를 검사한다.

2. 전체 시간표

시간	할 일	목표
0:00-0:05	수업 목표와 규칙 확인	오늘 만들 범위 이해
0:05-0:15	자유 주제 정하기	자기 프로젝트 방향 정하기
0:15-0:35	Node.js 확인과 설치	Codex CLI 설치 준비
0:35-0:50	Codex CLI 설치와 로그인	Codex 실행 성공
0:50-1:05	작업 폴더 만들기과 Plan Mode 시작	먼저 계획
1:05-1:15	계획 검토와 수정 요청	구조 승인
1:15-1:30	Codex가 미니 프로젝트 구현	파일 생성
1:30-1:40	Node 로컬 서버로 브라우저 확인	화면 열기
1:40-1:52	수동 확인과 test.js 실행	검증 체험
1:52-2:00	GitHub 저장소 연결과 업로드	결과 공유

3. 수업 규칙

- 사전 준비는 없다. 노트북과 인터넷만 있으면 시작한다.
- 참가자는 코드를 직접 고치지 않는다. 파일 생성과 수정은 Codex에게 맡긴다.
- 막히면 혼자 오래 붙잡지 않는다. 3분 이상 멈추면 진행자에게 알린다.

- 백엔드, 로그인, 데이터베이스, 결제, 외부 API는 쓰지 않는다.
- 개인정보, 비밀번호, 회사 내부 자료, API 키는 입력하지 않는다.
- 명령어 암기가 아니라 계획, 구현, 확인, 검증, 업로드의 흐름을 익힌다.

4. 진행 단계

1단계. 자유 주제 정하기 (0:05-0:15)

목표: 오늘 만들 웹 미니 프로젝트의 방향을 한 문장으로 정한다.

진행자 멘트

주제는 자유입니다. 단, 오늘은 전체 서비스를 만들지 않고 버튼 또는 입력창 하나가 있는 작은 화면만 만듭니다.

참가자가 따라 할 행동

- 종이나 메모장에 만들고 싶은 주제를 한 문장으로 쓴다.
- 주제가 크면 오늘 만들 수 있는 작은 화면으로 줄인다.
- 버튼 또는 입력창 중 어느 쪽이 자연스러운지 생각한다.

막히면 이렇게 처리

쇼핑몰은 상품 3개 소개 페이지로 줄인다.

예약 시스템은 예약 문의 버튼을 누르면 안내 문구가 나오는 페이지로 줄인다.

회원 기능, 결제, 로그인, 외부 API가 들어가면 오늘 범위에서 제외한다.

2단계. Node.js 확인과 설치 (0:15-0:35)

목표: Codex CLI 설치와 로컬 웹 확인을 위해 Node.js와 npm이 있는지 확인한다.

진행자 멘트

Node.js는 JavaScript를 내 컴퓨터에서 실행하게 해주는 프로그램입니다. npm은 Node.js와 같이 설치되는 도구 상자입니다.

참가자가 따라 할 행동

- Windows는 PowerShell, macOS는 Terminal을 연다.
- 진행자 화면을 보고 Node.js와 npm 버전 확인을 따라 한다.
- 버전이 나오면 설치를 건너뛴다.
- 버전이 안 나오면 브라우저에서 구글을 열고 nodejs를 검색한다.
- 공식 Node.js 사이트에 들어가 LTS라고 표시된 안정 버전을 설치한다.
- 설치가 끝나면 터미널을 완전히 닫고 다시 연 뒤 버전을 다시 확인한다.

막히면 이렇게 처리

설치했는데도 버전이 안 나오면 터미널을 새로 연다.

회사나 학교 컴퓨터라 설치가 막히면 진행자 화면을 보며 흐름을 따라간다.

3단계. Codex CLI 설치와 ChatGPT 로그인 (0:35-0:50)

목표: 터미널에서 Codex와 대화할 수 있는 상태를 만든다.

진행자 멘트

오늘은 ChatGPT 계정으로 로그인합니다. API key는 쓰지 않습니다. 설치 명령은 수업 당일 공식 문서를 보고 진행자 화면을 따라 합니다.

참가자가 따라 할 행동

- 진행자가 공식 Codex 설치 문서를 화면에 띄우는 것을 본다.
- 진행자 화면의 설치 명령을 PowerShell 또는 Terminal에 붙여넣고 실행한다.
- 설치가 끝나면 Codex 버전 확인을 진행자 화면대로 따라 한다.
- Codex를 실행한다.
- 브라우저가 열리면 ChatGPT 계정으로 로그인하고 승인한다.
- 터미널에 Codex 대화 화면이 보이면 성공이다.

막히면 이렇게 처리

오류가 나오면 지우지 말고 그대로 둔다.

npm 오류가 나오면 터미널을 새로 열고 다시 시도한다.

5분 이상 막히면 진행자 화면을 보며 Plan Mode 문장 작성부터 따라간다.

4단계. 작업 폴더 만들기 (0:50-0:55)

목표: Codex가 작업할 빈 폴더를 만든다.

진행자 멘트

오늘 폴더 이름은 영어와 숫자만 씁니다. 한글이나 공백은 처음 실습에서 헛갈릴 수 있습니다.

참가자가 따라 할 행동

- 바탕화면이나 문서 폴더 안에 새 폴더를 만든다.
- 폴더 이름은 my-codex-project처럼 단순하게 만든다.
- 터미널에서 그 폴더 위치로 이동한다.
- 현재 터미널 위치가 방금 만든 폴더인지 확인한다.

막히면 이렇게 처리

폴더 이동이 어렵다면 진행자가 OS별로 직접 도와준다.

폴더가 비어 있는 상태에서 시작하는 것이 가장 좋다.

5단계. Plan Mode로 계획 세우기 (0:55-1:05)

목표: Codex에게 바로 만들라고 하지 않고 먼저 계획만 세우게 한다.

진행자 멘트

지금부터는 만들지 않습니다. 먼저 계획만 세웁니다. 이 단계를 Plan Mode라고 부르겠습니다.

참가자가 따라 할 행동

- Codex에서 Plan Mode를 켤 수 있으면 진행자 안내에 따라 켜다.
- 아래 요청문을 자기 주제에 맞게 바꿔 보낸다.
- Codex가 바로 파일을 만들려고 하면 즉시 멈추고 계획만 다시 요청한다.

Codex에 보낼 문장

Plan Mode로 진행하자. 아직 파일을 만들거나 수정하지 말고, 먼저 계획만 세워줘. 내 주제는 [내 주제]야. 파일은 README.md, index.html, style.css, script.js, test.js로 구성하고, 백엔드, 로그인, 데이터베이스, 결제, 외부 API는 쓰지 않는다. 버튼 또는 입력창 중 하나만 사용하고, Node.js 로컬 서버와 test.js로 확인할 수 있게 계획해줘.

6단계. 계획 검토와 수정 요청 (1:05-1:15)

목표: 사람이 계획을 읽고 한 번 수정 요청한 뒤 구현을 승인한다.

진행자 멘트

Codex가 계획을 썼다고 바로 승인하지 않습니다. 오늘은 모두가 수정 요청을 한 번씩 합니다.

참가자가 따라 할 행동

- 내 주제가 화면에 분명히 드러나는지 확인한다.
- 파일 5개가 계획에 들어 있는지 확인한다.
- 버튼 또는 입력창 동작이 하나만 있는지 확인한다.
- 브라우저 확인과 테스트 방법이 포함되어 있는지 확인한다.
- 기능이 크거나 어려우면 더 작게 줄여달라고 요청한다.
- 계획이 적당하면 구현을 승인한다.

Codex에 보낼 문장

이 계획으로 진행하자. 이제 파일을 만들어줘. 구현이 끝나면 파일별로 무엇을 만들었는지 설명하고, 내가 브라우저에서 확인할 순서를 알려줘.

7단계. Codex가 프로젝트 구현 (1:15-1:30)

목표: Codex가 파일을 만들고 참가자는 파일 구성과 설명을 확인한다.

진행자 멘트

지금은 참가자가 코드를 직접 고치지 않습니다. Codex가 어떤 파일을 만들고 어떤 명령을 실행하는지 봅니다.

참가자가 따라 할 행동

- Codex가 작업하는 동안 파일 생성 과정을 본다.
- 구현이 끝나면 README.md, index.html, style.css, script.js, test.js가 있는지 확인한다.
- 빠진 파일이 있으면 Codex에게 빠진 파일을 만들어달라고 요청한다.
- 각 파일 역할을 한 문장씩 설명해달라고 요청한다.

8단계. 브라우저에서 확인 (1:30-1:40)

목표: VS Code 없이 Node.js 로컬 서버로 화면을 연다.

진행자 멘트

오늘은 VS Code를 쓰지 않습니다. Node.js로 간단한 로컬 서버를 열어서 확인합니다.

참가자가 따라 할 행동

- Codex에게 Node.js로 로컬 서버를 실행하는 방법을 OS에 맞게 안내해달라고 요청한다.
- Codex가 알려준 순서대로 로컬 서버를 실행한다.
- 터미널에 나온 주소를 브라우저에 입력한다.
- 자기 주제, 버튼 또는 입력창, 화면 변화가 보이는지 확인한다.

Codex에 보낼 문장

이 프로젝트를 VS Code 없이 브라우저에서 확인하고 싶어. Node.js로 간단한 로컬 서버를 실행하는 방법을 내 OS에 맞게 안내해줘. 실행 후 어떤 주소를 브라우저에서 열어야 하는지도 알려줘.

막히면 이렇게 처리

페이지가 안 열리면 터미널 메시지와 브라우저 메시지를 지우지 말고 Codex에게 보여준다.

버튼이 안 되면 전체를 다시 만들지 말고 그 문제만 고쳐달라고 요청한다.

9단계. 수동 테스트와 test.js 실행 (1:40-1:52)

목표: 눈으로 확인하고, Node.js 검사 파일도 실행한다.

진행자 멘트

보이니까 끝이 아닙니다. 직접 눌러보고, test.js도 실행해 확인합니다.

참가자가 따라 할 행동

- 페이지 제목과 내 주제가 보이는지 확인한다.
- 버튼 또는 입력창이 보이는지 확인한다.
- 버튼을 누르거나 입력했을 때 화면이 바뀌는지 확인한다.
- 브라우저 창을 좁혀도 내용이 심하게 겹치지 않는지 확인한다.
- 새로고침하면 기본 화면으로 돌아오는지 확인한다.
- Codex에게 test.js 실행 방법을 묻고 그대로 실행한다.
- 실패 메시지가 나오면 그대로 Codex에게 보여주고 작은 수정만 요청한다.

Codex에 보낼 문장

test.js가 README.md, index.html, style.css, script.js 존재 여부, CSS/JS 연결 여부, 버튼 또는 입력창 존재 여부, README 설명과 실행 방법을 확인하는지 봐줘. 부족하면 test.js를 수정하고 실행 방법을 알려줘.

10단계. GitHub에 올리기 (1:52-2:00)

목표: Git 명령을 외우지 않고 Codex에게 저장소 연결, 커밋, push, 확인을 맡긴다.

진행자 멘트

GitHub은 오늘 만든 프로젝트를 올려두는 저장소로만 사용합니다. Git 명령은 Codex에게 맡기되, 무엇을 하려는지는 확인합니다.

참가자가 따라 할 행동

- Codex에게 Git 설치 여부를 확인해달라고 요청한다.
- Git이 없으면 구글에서 git을 검색해 공식 사이트에서 설치한다.
- 브라우저에서 GitHub에 로그인하거나 계정을 만든다.
- 빈 저장소를 만들고 저장소 주소를 복사한다.
- Codex에게 현재 폴더를 저장소와 연결하고 첫 커밋과 push를 진행해달라고 요청한다.
- GitHub 웹페이지에서 파일 5개가 올라갔는지 확인한다.

Codex에 보낼 문장

이 폴더를 내가 만든 GitHub 저장소와 연결하고 싶어. 저장소 주소는 [주소]야. 먼저 현재 폴더와 파일 상태를 확인하고, 어떤 명령을 실행할지 설명해줘. 내가 확인하면 첫 커밋을 만들고 GitHub에 push한 뒤 정상 업로드됐는지 확인해줘.

막히면 이렇게 처리

시간이 부족하면 저장소 생성까지만 하고 다음에 이어갈 요약을 Codex에게 요청한다.

인증 화면이 열리면 브라우저에서 GitHub 로그인을 완료하고 터미널로 돌아온다.

5. 마무리 회고

진행자는 마지막 1분에 아래 질문을 빠르게 확인한다.

- Codex에게 계획을 먼저 세우게 했는가?
- 구현 전에 계획을 한 번 수정 요청했는가?
- 브라우저에서 직접 확인했는가?
- test.js를 실행했는가?
- GitHub 업로드까지 갔는가, 아니면 어디서 멈췄는가?

6. 진행자 탈출 조건

- Node 설치가 막힌 사람은 진행자 화면을 보며 Plan Mode 문장만 작성한다.
- Codex 로그인 실패자는 진행자 화면을 따라 하며 주제와 계획 검토를 함께 한다.
- 구현이 늦어지면 디자인 요구를 줄이고 버튼 또는 입력창 하나만 남긴다.
- 브라우저 확인이 늦어지면 test.js보다 수동 확인을 우선한다.
- GitHub가 늦어지면 저장소 생성까지만 하고 다음 작업 요약을 만든다.